GIAO DỊCH VÀ ACID

Chuyên đề minh họa bằng chuyển tiền và lịch đồng thời

Mã tài liệu: CD | Dữ liệu và số tiền đều là giả lập.

Giả định

Có hai tài khoản: A có 1000, B có 500. Chuyển 100 từ A sang B không mất phí. Tổng trước và sau là 1500. Các ví dụ đơn phiên không có thao tác khác xen vào, trừ phần nêu lịch đồng thời.

Các mục

CD-1: thiết lập dữ liệu và chuyển tiền có kiểm tra. CD-2: nguyên tử và nhất quán. CD-3: cô lập và ba lịch bất thường. CD-4: bền vững và kết thúc giao dịch. CD-5: khóa dòng với hai phiên PostgreSQL.

> Đây là ví dụ học tập về giao dịch. Chỉ chạy trong CSDL thử nghiệm; không dùng như một hệ thống thanh toán hoàn chỉnh.

CD-1 Chuyển tiền có kiểm tra

```
CREATE TABLE TaiKhoan (
  MaTK VARCHAR(1) PRIMARY KEY,
  SoDu NUMERIC(12,2) NOT NULL CHECK (SoDu >= 0)
);
INSERT INTO TaiKhoan VALUES ('A',1000),('B',500);
```

Ứng dụng phải kiểm tra mã gửi khác mã nhận, số tiền dương và hai tài khoản tồn tại. Ví dụ này chuyển 100 từ A sang B. Trình tự dưới đây gồm câu SQL và bước kiểm tra của ứng dụng; không phải một đoạn SQL chạy liên tục không điều kiện.

```
BEGIN;
SELECT MaTK, SoDu FROM TaiKhoan
WHERE MaTK IN ('A','B')
ORDER BY MaTK FOR UPDATE;
```

Ứng dụng chỉ tiếp tục nếu nhận đúng hai dòng và số dư A ít nhất 100. Nếu thiếu tài khoản hoặc thiếu số dư, chạy ROLLBACK và kết thúc thao tác. Nếu đủ điều kiện, thực hiện hai câu sau và xác nhận mỗi câu tác động đúng một dòng.

```
UPDATE TaiKhoan SET SoDu = SoDu - 100
WHERE MaTK = 'A';
UPDATE TaiKhoan SET SoDu = SoDu + 100
WHERE MaTK = 'B';
COMMIT;
```

Khi cả hai cập nhật thành công và chốt xong, A = 900, B = 600, tổng 1500. Nếu bất kỳ bước kiểm tra hoặc SQL nào thất bại trước COMMIT, ứng dụng phải ROLLBACK toàn bộ giao dịch.

> UPDATE không tìm thấy dòng có thể kết thúc thành công với số dòng tác động bằng 0. Không được xem “không có lỗi SQL” là đủ để kết luận chuyển tiền thành công.

CD-2 Nguyên tử và nhất quán

Tình huống 1: dừng giữa hai bước

Bắt đầu A = 1000, B = 500. T1 đã trừ A xuống 900 nhưng chưa cộng B. Nếu T1 phát hiện lỗi và ROLLBACK trước khi chốt, kết quả phải trở về A = 1000, B = 500. Không để lại trạng thái A = 900, B = 500 như kết quả cuối của giao dịch bị hủy. Đây là tính nguyên tử.

Tình huống 2: thiếu quy tắc nghiệp vụ

Giả sử mã người nhận B bị nhập nhầm thành Z không tồn tại. UPDATE tài khoản Z có thể tác động 0 dòng mà không phát sinh lỗi SQL. Nếu ứng dụng vẫn COMMIT phần trừ A, tổng tiền hai tài khoản giảm từ 1500 xuống 1400. Giao dịch có thể hoàn tất về mặt câu lệnh nhưng sai quy tắc chuyển tiền.

Ràng buộc SoDu >= 0 chỉ chặn số dư âm. Nó không tự kiểm tra việc tiền đã được cộng vào một tài khoản đích hợp lệ. Tính nhất quán cần kết hợp ràng buộc CSDL với điều kiện và trình tự của ứng dụng.

Tiêu chí	Nguyên tử	Nhất quán
Câu hỏi	Có giữ lại thay đổi từng phần khi giao dịch thất bại không?	Trạng thái cuối có thỏa quy tắc đã xác định không?
Ví dụ đúng	Hủy cả trừ và cộng nếu thao tác không hoàn tất.	Chuyển 100 không phí giữ tổng 1500 và số dư không âm.

Kết luận chuẩn: nguyên tử bảo vệ phạm vi “tất cả hoặc không có gì”; nhất quán đòi hỏi các thao tác trong phạm vi đó thực sự bảo toàn các quy tắc dữ liệu.

CD-3 Đọc lịch thao tác đồng thời

Bước	Lost update - ví dụ đọc rồi ghi giá trị cũ
1	T1 đọc A = 1000; T2 cũng đọc A = 1000.
2	T1 tính 1000 - 100 và ghi A = 900; T1 COMMIT.
3	T2 dùng bản đọc cũ tính 1000 + 50 và ghi A = 1050; T2 COMMIT.
4	Đúng theo cả hai thay đổi phải là 950; thực tế 1050 làm mất tác dụng trừ 100.

Bước	Dirty read - lịch khái niệm
1	T1 ghi A = 900 nhưng chưa COMMIT.
2	T2 đọc giá trị 900 chưa được T1 chốt.
3	T1 ROLLBACK về 1000; T2 đã dùng dữ liệu chưa chốt.

Bước	Non-repeatable read
1	T2 bắt đầu giao dịch, đọc A = 1000.
2	T1 cập nhật A = 900 và COMMIT.
3	T2 đọc lại A và thấy 900 trong cùng giao dịch T2.

Dấu hiệu phân biệt: dirty read dùng thay đổi chưa chốt của giao dịch khác; non-repeatable read thấy thay đổi đã chốt giữa hai lần đọc. Lost update nhấn mạnh việc ghi đè cập nhật từ giá trị đọc cũ.

Lịch dirty read ở đây chỉ dùng để giải thích khái niệm. PostgreSQL xử lý READ UNCOMMITTED như READ COMMITTED nên không tái hiện đọc bẩn của giao dịch khác. Lịch non-repeatable read có thể minh họa bằng hai phiên ở READ COMMITTED.

CD-4 Chốt, hủy và tính bền vững

COMMIT kết thúc giao dịch thành công bằng việc chấp nhận các thay đổi. ROLLBACK hủy phần thay đổi chưa chốt trong giao dịch hiện tại. Hai lệnh không phải nút bật tắt có thể dùng để đảo qua đảo lại một thay đổi đã chốt.

Thời điểm	A	B	Diễn giải
Ban đầu	1000	500	Trạng thái đã có.
Trong giao dịch	900	600	Các thay đổi chưa được chốt.
Nếu ROLLBACK	1000	500	Hủy toàn bộ thay đổi của giao dịch.
Nếu COMMIT	900	600	Chốt kết quả thành công.

Ví dụ nhận diện durability

Nếu hệ thống đã xác nhận COMMIT thành công cho chuyển tiền, sau một lần khởi động lại thuộc phạm vi bảo đảm bền vững, A và B vẫn phải phản ánh kết quả đã chốt. Khả năng phục hồi kết quả đã chốt minh họa tính bền vững, không phải tính cô lập.

Bền vững phụ thuộc cấu hình và giả định về loại sự cố. Nó không thay thế sao lưu. Một bản sao lưu còn cần thiết để xử lý những tình huống mất dữ liệu nằm ngoài phạm vi một lần khởi động lại thông thường.

Lỗi sau khi đã chốt

Nếu người dùng nhập sai số tiền nhưng giao dịch đã COMMIT, ROLLBACK của một giao dịch mới không tự xóa giao dịch cũ. Cần một nghiệp vụ điều chỉnh hoặc bù trừ hợp lệ, có kiểm tra và lưu dấu vết theo yêu cầu của ứng dụng.

> Trong demo, dùng truy vấn đọc số dư trước và sau COMMIT hoặc ROLLBACK để minh họa. Không cần mô phỏng phá hủy dữ liệu hay tắt máy đột ngột để chấm câu hỏi về ACID.

CD-5 Minh họa khóa bằng hai phiên

Chuẩn bị hai phiên kết nối vào cùng CSDL thử nghiệm. Khôi phục A = 1000, B = 500 trước khi bắt đầu và bảo đảm chưa có giao dịch cũ đang mở.

Phiên 1: khóa A và để giao dịch mở

```
BEGIN;
SELECT SoDu FROM TaiKhoan
WHERE MaTK = 'A' FOR UPDATE;
```

Phiên 2: thử cập nhật cùng dòng

```
BEGIN;
UPDATE TaiKhoan SET SoDu = SoDu + 50
WHERE MaTK = 'A';
```

Câu UPDATE ở phiên 2 chờ khóa xung đột. Trong khi nó đang chờ, quay lại phiên 1 và chạy COMMIT. Sau khi phiên 1 giải phóng khóa, phiên 2 có thể hoàn tất cập nhật. Cuối cùng chạy COMMIT ở phiên 2. Nếu không có thao tác khác, A trở thành 1050.

Điểm cần quan sát

FOR UPDATE bảo vệ dòng khỏi các yêu cầu khóa hoặc ghi xung đột cho tới khi giao dịch kết thúc. Truy vấn SELECT thông thường ở một phiên khác có thể vẫn đọc phiên bản dữ liệu phù hợp mà không bị chặn bởi khóa dòng này.

Để giảm thời gian chờ, chỉ giữ khóa trong khoảng cần kiểm tra và cập nhật. Với giao dịch cần khóa cả A lẫn B, các luồng nên lấy khóa theo cùng một thứ tự, ví dụ thứ tự mã tài khoản.

> Đây là lịch kiểm thử thao tác đồng thời cần chạy trên PostgreSQL. Không lấy kết quả của SQLite hay một phiên giả lập duy nhất để kết luận đã kiểm chứng hành vi khóa PostgreSQL.